

Version 4.1

User Manual

Publication Date: 30-Oct-2026

Powered by the miniaudio audio library.



ALTEA AEROSPACE

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Web: http://www.alteaaerospace.com
Email: mailto:doug.beachy@outlook.com
Orbiter Forum: dbeachy1 (http://orbiter-forum.com)

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Table of Contents

Copyright 3
Summary 4
XRSound Features.....5
Requirements.....9
Installation 9
Configuring XRSound.....11
Adding Custom Sounds to a Vessel via a Config File.....11
Using the XRSound C++ APIs to Add Sounds to a Vessel.....14
Changing Volume Levels and Settings at Runtime 17
Using the XRSound C++ APIs to Add Sounds to a Module.....19



The Future Is Now.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Copyright

This software copyright 2018-2026 Douglas Beachy and contributors. This software is open-source and licensed under the MIT license.

MIT License

Copyright (c) 2018-2026 Douglas E. Beachy and contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.

XRSound includes the miniaudio library by David Reid and the stb_vorbis Ogg Vorbis decoder by Sean Barrett. Each is offered under a choice of public domain or a permissive license (MIT No Attribution for miniaudio, MIT for stb_vorbis); see the license notices at the end of miniaudio.h and stb_vorbis.c in the XRSound source.

Summary

XRSound provides over 450 fully configurable, default sounds and voice callouts for all Orbiter vessels. In addition, it provides a C++ API for add-on Orbiter vessel and plugin authors to use via the XRSound.dll Orbiter plugin module.

XRSound is fully configurable, and in addition to the global `XRSound.cfg` file that configures default sounds for all vessels, you can also edit or create Orbiter vessel class-specific config files, such as the included `XRSound-DeltaGlider.cfg`, etc., that assign default sounds to vessel-specific animations and events without requiring recompiling the vessel or doing any coding.

Special thanks to Grzegorz Lorens (Loru on Orbiter-Forum) for creating the XRSound logo and a brand-new ambient music track that by default plays in external views in space (details below).

XRSound 4.0 replaced the irrKlang sound engine with the open-source miniaudio library; thanks to Gondos for this work. XRSound no longer needs irrKlang.dll or the ikpMP3.dll and ikpFlac.dll plugins.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Each vessel or module can change its own volume levels and settings while the simulation runs.

XRSound Features

- Automatically adds default sounds for any Orbiter vessel that defines thrusters -- not just XR vessels. You can also manually assign sounds to any vessel class via its `XRSound-<vessel class>.cfg` file; e.g., `XRSound-DeltaGlider.cfg`.

- Add-on developers can play custom sounds from their Orbiter *modules* (e.g., MFDs or other plugins), not just Orbiter vessels.

- Each new release of XRSound is backwards-compatible with all existing XRSound-enabled vessels.

- Each sound or group of sounds is fully configurable via XRSound.cfg and optional Orbiter vessel class-specific config files, or via the included XRSound SDK.

- Includes over 450 default sound files, including all XR vessel voice callouts by actress Sally Beaumont and 141 ATC callouts from STS-114 and STS-121 featuring CAPCOM Julie Payette. These ATC callouts were recorded live from NASA TV and saved as 48 kHz, 16-bit mono.

- Add-on authors are free to use any of the sounds in their own vessels.

- Includes new *Solar Serenity* ambient music track created by and licensed from Grzegorz Lorens ("Loru" on Orbiter-Forum) that plays in

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

external views in space by default. This is configurable like any other sound. (For a lossless version of Loru's *Solar Serenity*, see here.)

- All default sounds are fully configurable, including:
  - Air conditioning
  - Audio greeting voice callout on simulation start
  - Music in external views in space
  - Wind effects while landed in an atmosphere
  - Main engine sound
  - Hover engine sound
  - Retro engine sound
  - RCS sounds
  - Switch on / Switch off clicks for RCS and AF Ctrl changes
  - RCS mode change voice callouts ("Rotation", "Translation", "Off")
  - AF Ctrl mode voice callouts ("Off", "Pitch", "On")
  - Crash sound
  - Hard landing sound
  - Tire chirp on touchdown for vessels that define landing gear animation ID via their `XRSound-<class name>.cfg` file. [You can set `` `LogVesselAnimations = 1` `` in XRSound.cfg to log the animation IDs for all vessels as you activate each animation.]
  - Tires rolling sound; volume varies by ship velocity
  - One additional custom engine sound per vessel (e.g., the SCRAM engines in the stock DeltaGlider-S). Configurable via a given vessel class's `XRSound-<vessel class>.cfg` file.
  - Wheel brakes sound
  - Takeoff and landing voice callouts ("100 knots", "Wheels up" / "Liftoff", "You are cleared to land", "Warning: gear is up", "Touchdown", "Wheel stop", etc.)

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

    - Wind sound and plasma sound in flight; volume varies by dynamic pressure and distance
    - Autopilot on / off tones
    - Ambient sounds configurable by pointing to a folder in XRSound.cfg; twelve ambient sounds are included by default, but there is no limit to the number of ambient sounds supported. Minimum and maximum playback interval is configurable.
    - Fully configurable music folder support: music files may be played back sequentially or at random (or be disabled) and may be configured to play internally / externally / in space / while landed, or any combination thereof.
    - Altitude voice callouts
    - Docking distance voice callouts
    - Docking and undocking voice callouts and sounds
    - Docking radar sounds whose beep interval varies by the distance to the docking port
    - Mach voice callouts ("Mach 1", "Mach 2", "Mach 27 Plus", "Subsonic", etc.)
    - ATC sounds played at random at configurable intervals from a specific folder. Like all other sounds and groups, the folder can be changed at runtime via an SDK call and may be configured via XRSound.cfg or `XRSound-<vessel class>.cfg` otherwise.

- The new XR vessel versions use XRSound, and therefore no longer need to bundle their sound files with each vessel anymore: all sounds are included with the XRSound download.

- Can add sound events to any existing Orbiter vessel's animations simply by editing that vessel's class's `XRSound-<classname>.cfg` file. Includes .cfg files for each of the default Orbiter vessels.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

- Includes C++ SDK for add-on authors to use in their own Orbiter vessels or Orbiter modules; XR vessels use it.

- Vessels and modules can change their own main volume, the volume of each type of sound, the volume of each individual sound, and most XRSound.cfg settings at runtime via the XRSound SDK, without affecting any other vessel or module.

- Users can replace or disable any or all of the default sounds via the XRSound SDK or `XRSound.cfg`.

- Orbiter add-ons that link with XRSound.lib can still load and run (without sounds, of course) even if the user does not have XRSound installed.

- No limit to the number of sound slots that vessels may use.

- No limit to the number of vessels or modules in the simulation (beyond available physical memory, of course).

- No limit to the number of ATC radio chatter files for a given "radio frequency" (i.e., folder): just configure the ATC folder via your `XRSound.cfg` file, and all sound files in that folder are played at random for ATC chatter.

- No limit to the number of music files in the configured music folder.

- Parameters for minimum and maximum times between ATC chatter playback are configurable via `XRSound.cfg`.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

- Uses the miniaudio library, which supports the following sound file formats with no plugin DLLs:
  - RIFF WAVE (*.wav)
  - Ogg Vorbis (*.ogg)
  - MPEG-1 Audio Layer 3 (*.mp3)
  - Free Lossless Audio Codec (*.flac)

Refer to the comments in `$ORBITER_ROOT\XRSound\XRSound*.cfg` files for full details about configuring XRSound.

Refer to the comments in `XRSound.h` (used with static library XRSound.lib) in `$ORBITER_ROOT\Orbitersdk\XRSound` for more information about the XRSound API.

Requirements

- Windows Vista or newer.
- Orbiter 2024 or newer.
- Microsoft Visual C++ Redistributable files (latest version).

Installation

This section details how to install and activate XRSound. Note that XRSound is a plug-in for *Orbiter* and requires that Orbiter 2024 or newer be installed first.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

1. Install Orbiter 2024 or newer. Older versions of Orbiter are not supported.

2. Install the x86 (for 32-bit Orbiter) or x64 (for 64-bit Orbiter) Microsoft Visual C++ Redistributable files (latest version).

3. XRSound is included with Orbiter 2024 and newer. If you are installing a separate XRSound release, unzip it into your `$ORBITER_ROOT` (e.g., C:\Orbiter) directory.

4. Bring up Orbiter to display the Orbiter Launchpad.

5. Click the *Modules* button.

6. Click the checkbox next to *XRSound* in the *Sound module for Orbiter* section to activate XRSound.

   Note: if you previously installed the OrbiterSound module, deactivate it by unchecking it in the same section. Otherwise, its default vessel sounds will conflict with XRSound's default vessel sounds. However, you can install Face's SoundBridge module to seamlessly route OrbiterSound calls to XRSound if you have vessels installed that require OrbiterSound.

Now run Orbiter and load any of the launch scenarios; you should hear an audio greeting voice callout when the simulation starts. If you don't, look at `$ORBITER_ROOT\XRSound.log` for error or warning messages. Sound engine and sound file errors are also written to `$ORBITER_ROOT\Orbiter.log`.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Configuring XRSound

XRSound is fully configurable and allows users to replace or disable any sound or voice callout either globally, via the `$ORBITER_ROOT\XRSound\XRSound.cfg` file, or on a per-vessel-class basis via `$ORBITER_ROOT\XRSound\XRSound-<vessel class name>.cfg` file (for example `C:\Orbiter\XRSound\XRSound-DeltaGlider.cfg`). Settings in the vessel class's .cfg file override any corresponding settings in the global `XRSound.cfg` file.

Refer to the comments in `$ORBITER_ROOT\XRSound\XRSound.cfg` for details about how to configure XRSound.

Adding Custom Sounds to a Vessel via a Config File

There are two ways to define custom sounds and/or voice callouts to an existing vessel. The first is to use the XRSound C++ API and make XRSound calls from the vessel's source code. This allows you full access to all XRSound's features, but also requires you to have access to the vessel's source code.

The second way is to create or edit an `$ORBITER_ROOT\XRSound\XRSound-<className>.cfg` file for the vessel's class in Orbiter. In addition to defining (or "overriding") any default sounds defined in XRSound.cfg, you can also define new sounds for any of the vessel's animations (for example, a nosecone opening or closing), as well as define one custom engine sound (for example, SCRAMJETS on the default DeltaGlider-S). XRSound includes custom .cfg files for each of Orbiter's default vessels. For example, the

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

DeltaGlider-S's Orbiter class name is "DG-S", and so its custom XRSound.cfg file is named XRSound-DG-S.cfg.

Note: if a vessel's Orbiter class name contains any characters that are invalid filename characters (e.g., "/"), they will be converted to underscores. For example, a vessel class named `Foo/BarClass` *becomes* `Foo_BarClass`*, so its XRSound config file would be* `XRSound-Foo_BarClass.cfg`*.*

Here is how you would typically go about adding some new sounds to an existing vessel via its XRSound config file:

1. Edit `$ORBITER_ROOT\XRSound\XRSound.cfg` and make these edits:

   ```
   EnableVerboseLogging = 1
   LogVesselAnimations = 1
   LogThrusterData = 1
   ```

   Note: These settings should only be enabled temporarily while you are doing your XRSound development, because having that extra logging enabled can greatly increase the rate at which the `$ORBITER_ROOT\XRSound.log` file grows!

2. Run Orbiter and launch a scenario with the vessel whose sounds you want to customize.

3. Activate the vessel animation you want to define sounds for (for example, deploy the landing gear with the "G" key).

4. Look in XRSound.log for lines like this:

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

```
[GL-01]   >> LogVesselAnimations:
[DeltaGlider][animation ID = 7, state = Moving]
```

That line shows that the landing gear animation for the vessel named `GL-01` with Orbiter vessel class `DeltaGlider` has the Orbiter-assigned animation ID of `7`.

5. Since the vessel class is `DeltaGlider`, the file you need to edit to assign custom sounds to it is `$ORBITER_ROOT\XRSound\XRSound-DeltaGlider.cfg`. Open that file in your favorite text editor. (If the file does not exist yet, copy an existing vessel's cfg file as a template and then delete any existing settings in it before defining new ones.)

6. Add or edit sounds for the animation events for the animation ID you are interested in (e.g., ID 7 for the DeltaGlider's landing gear). Refer to the comments in `$ORBITER_ROOT\XRSound\XRSound-DeltaGlider.cfg` for details about when each custom animation sound and/or thruster sound plays. An animation section can also set Volume, from 0 (silent) to 1.0 (full, the default), to play that animation's sounds more quietly; for example, the Shuttle-A's rear thruster pods use Volume = 0.3.

7. Don't forget to reset your logging settings back to zero in `$ORBITER_ROOT\XRSound\XRSound.cfg` once you are done adding and debugging your sounds:

```
EnableVerboseLogging = 0
LogVesselAnimations = 0
LogThrusterData = 0
```


Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

Using the XRSound C++ APIs to Add Sounds to a Vessel

XRSound includes a simple, easy-to-use API to play and manipulate sounds from a vessel. These SDK files are installed to `$ORBITER_ROOT\Orbitersdk\XRSound`:

`XRSound.h`	Defines and documents the API methods
`XRSound.lib`	C++ static library

Link with `XRSound.lib`. Orbiter's build creates `XRSound.lib` under that one name for both Debug and Release builds, so link with the copy from an Orbiter SDK built in the same configuration (Debug or Release) as your add-on, using the Visual Studio version that built that SDK or a newer one. This library is lightweight and at runtime simply dynamically invokes the corresponding methods in `XRSound.dll`, if present.

For full details about the XRSound C++ API methods, refer to the detailed comments in `$ORBITER_ROOT\Orbitersdk\XRSound\XRSound.h`. Here are some simple examples of how to use the XRSound API in your custom vessel:

```
#include "XRSound.h"

// could use #define for your custom sound IDs instead since
sound IDs are just signed integers, but an enum is cleaner
enum MySounds
{
    SystemReset, // value 0
    SomeOtherSound,
```


Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

```
    …
};

class MyVessel : public VESSEL4
{
    ...
    XRSound *m_pXRSound;
    bool m_bResettingTheSystem;
};

void MyVessel::clbkPostCreation()
{
    m_pXRSound = XRSound::CreateInstance(this);   // create sound
engine instance for this vessel

    // load a custom sound for this vessel
    m_pXRSound->LoadWav(SystemReset, "XRSound\\Default\\System
Reset.wav", XRSound::Radio);  // returns false if file not found

    // disable the default "100 knots" voice callout
    m_pXRSound->SetDefaultSoundEnabled(XRSound::OneHundredKnots,
false);

    // replace the default docking voice callout with a custom
one that resides in $ORBITER_ROOT\MySoundsFolder
    m_pXRSound->LoadWav(XRSound::DockingCallout,
"MySoundsFolder\\My custom docking callout.mp3", XRSound::Radio);
}

void MyVessel::clbkPreStep(double simt, double simdt, double mjd)
{
    if (m_bResettingTheSystem)
```


Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

```
    {
        // play our custom sound: don't loop it, and use max
volume
        m_pXRSound->PlayWav(SystemReset, false, 1.0);   //
returns false if play fails
        m_bResettingTheSystem = false;  // so we don't keep
looping it (although calling PlayWav if the sound is already
playing can only change its volume or loop settings: it will not
stop and restart it).
    }
}

MyVessel::~MyVessel()
{
    // as with any other allocated member variables, always
remember to clean up the XRSound engine instance in your vessel's
destructor! :)
    delete m_pXRSound;
}
```

Of course, you can also load a new sound into the same sound slot at any time; if a sound is already playing in that slot, it will automatically be stopped when LoadWav replaces it:

```
if (m_pXRSound->LoadWav(SystemReset, "My new sound.ogg",
XRSound::Radio))   // does sound file exist?
    m_pXRSound->PlayWav(SystemReset);    // play the sound using
defaults of loop = false and max volume
```

One important thing to note regarding performance is that `LoadWav` is lightweight and does not actually *load* the sound data into memory: it simply

1) verifies that the sound file exists and is readable, and

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

2) saves the file path and playback type in the master sound map in memory for this vessel.

Sound data is only loaded when it is played: XRSound streams each sound from its file rather than loading the whole file into memory, and releases it automatically when it is no longer needed. This is managed by the miniaudio library.

Note that XRSound does not write any data to or read any data from Orbiter scenario files.

Changing Volume Levels and Settings at Runtime

Every vessel and every module has its own volume levels and its own copy of each XRSound setting. When your vessel or module creates its XRSound instance, they start at the values configured in XRSound.cfg (and, for a vessel, in its XRSound-<vessel class>.cfg file). Your code can read and change them at any time. A change affects only your own vessel or module: it never changes any XRSound config file or any other vessel or module.

To read or change a volume level, call `GetVolume` or `SetVolume` with an `XRSound::VolumeType` and a level from 0 (muted) to 1.0 (full). The volume types are:

- `Main`: everything this vessel or module plays. Starts at MasterVolume in XRSound.cfg.
- `Engines`: main, retro, hover, and custom engine sounds.
- `RCS`: RCS thruster sounds.
- `Effects`: sound effects such as switches, wind, tires, docking, and crashes, plus the vessel's door and other animation sounds.
- `Callouts`: voice callouts.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

- `CabinAmbience`: cabin ambience and air conditioning.
- `ATC`: ATC radio chatter. Starts at ATC Volume in XRSound.cfg.
- `Music`: the music level while this vessel has focus. Starts at MusicVolume in XRSound.cfg.
- `Custom`: sounds that your code loads with its own sound IDs.

Unless noted above, each level starts at 1.0. A sound's final volume is its own volume times the level for its type times the Main level, before XRSound fades it for distance and atmosphere.

You can also set the volume of a single sound with `SetVolume(soundID, volume)`. For a sound you loaded, this is the volume that `PlayWav` uses when you do not pass one. For a default sound, it scales the volume that XRSound picks for it automatically (for example, by thrust level).

To read or change a setting, call `GetSetting` or `SetSetting` with an `XRSound::Setting`. The settings have the same names and valid values as in XRSound.cfg, such as ATCMinDelay, MusicOrder, and SilenceOfSpace; see XRSound.h for the full list and when each change takes effect.

`GetDefaultVolume` and `GetDefaultSetting` return the configured starting value for your vessel or module, which never changes during the session. Use them for things like a "reset to default" button.

`SetPlaybackType` and `GetPlaybackType` change how a single sound is faded, and `GetDuration` gets a sound file's length without playing it. `SetPan`, `SetPlaybackSpeed`, and `SetPlayPosition` also work on a loaded sound before it starts playing: the value is applied the next time `PlayWav` starts it.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

If the user's XRSound.dll is older than 4.1, `SetVolume`, `GetVolume`, `SetSetting`, `GetSetting`, `GetDefaultVolume`, `GetDefaultSetting`, `SetPlaybackType`, `GetPlaybackType`, and `GetDuration` return false, so your add-on still loads and runs with older XRSound versions.

Here is an example:

```
// turn this vessel's engine sounds down to half
m_pXRSound->SetVolume(XRSound::VolumeType::Engines, 0.5f);

// play ATC chatter less often for this vessel only
m_pXRSound->SetSetting(XRSound::Setting::ATCMinDelay, 60);
m_pXRSound->SetSetting(XRSound::Setting::ATCMaxDelay, 300);

// "reset to default" button for this vessel's main volume
float level;
if (m_pXRSound->GetDefaultVolume(XRSound::VolumeType::Main,
level))
    m_pXRSound->SetVolume(XRSound::VolumeType::Main, level);
```

Using the XRSound C++ APIs to Add Sounds to a Module

As of XRSound version 2.0, you can also play sounds from an Orbiter *module,* such as an MFD or other type of Orbiter plugin. Adding sounds to a module is exactly the same as adding sounds to a vessel except for the following differences:

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

- You create an instance of an XRSound object that is tied to an arbitrary ID, rather than an Orbiter VESSEL object, by calling the `XRSound::CreateInstance(const char *pUniqueModuleName)` method from your module's `clbkSimulationStart` method.

  Your module should *always* call `XRSound::CreateInstance` from your module's `clbkSimulationStart` method, and it should also delete the returned `XRSound` object in your module's `clbkSimulationEnd` method. Note that XRSoundEngine objects are only valid until `clbkSimulationEnd` finishes.

  Note: Since XRSound uses the `pUniqueModuleName` value to uniquely identify which Orbiter module is calling it, it is best to pass the name of your Orbiter module's DLL here; e.g., if your custom MP3 Orbiter module is installed as `MyCoolMp3Player.dll`, it's best to pass "MyCoolMp3Player" as the `pUniqueModuleName` above. Technically, `pUniqueModuleName` values are arbitrary, but they must be unique within a given Orbiter installation.

- When calling `LoadWav(const int soundID, const char *pSoundFilename, const PlaybackType playbackType)`, you should pass `XRSound::PlaybackType::Global` for `playbackType`, since other values have no effect when playing sounds from an Orbiter module (as opposed to an Orbiter vessel).

- Unlike Orbiter vessels, Orbiter modules do not have the concept of default sounds or default sound groups. Therefore, for XRSound objects created via `XRSound::CreateInstance(const char *pUniqueModuleName)`:

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

    - `SetDefaultSoundEnabled` has no effect and always returns `false`.
    - `GetDefaultSoundEnabled` always returns `false`.
    - `SetDefaultSoundGroupFolder` has no effect and always returns `false`.
    - `GetDefaultSoundGroupFolder` always returns `nullptr`.

- Modules have their own volume levels and settings too (see Changing Volume Levels and Settings at Runtime above). A module can set the `Music` level and the vessel-related settings, but they have no effect for a module, because modules do not play music or default sounds.

- There is no way to add sounds to an Orbiter module via a config file: you must use the XRSound C++ API from the module's source code.

Refer to `$ORBITER_ROOT\Orbitersdk\XRSound\XRSound.h` for more information about the XRSound SDK. For questions or support regarding XRSound, visit http://www.orbiter-forum.com.

Audio debug window (developers)

Debug builds of XRSound include an Audio debug window that lists each active sound, whether it is playing, paused, or stopped, whether it loops, and its file name. Open it from Orbiter's Custom Functions list (Ctrl+F4) by selecting Audio debug.

The window uses Orbiter's ImGui dialogs, which Orbiter added after the Orbiter 2024 release, so a Debug `XRSound.dll` that includes the window does not load in Orbiter 2024. To build a Debug XRSound for Orbiter 2024, configure CMake with `-DXRSOUND_DEBUG_GUI=OFF`. Release builds never include the window.

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.

-- end --

Copyright 2018-2026 Douglas Beachy and contributors. Licensed under the MIT license.